

Projeto Portfólio — Desenvolvendo Software com IA Generativa

Disciplina: Mod4 / PPI — Desenvolvendo Software com IA Generativa

Formato: em duplas (2 alunos por projeto)

Quando: Dia 3 da disciplina (08/06/2026), 10:00–17:00

Apresentação prévia: Dia 2, slot 16:15–16:45

Peso na nota: 60% da disciplina

Entrega: URL pública da demo + URL do repositório + URL do vídeo demo (≤ 3 min)

1. Visão geral

O Projeto Portfólio integra **tudo o que foi praticado em Dias 1 e 2**: prompt engineering, function-calling (tool-use), pipeline RAG ponta-a-ponta, avaliação com RAGAS, cache semântico, model routing e deploy em URL pública. O artefato final é um produto LLM-powered apto a entrar em **portfólio público** — exibível a recrutadores, clientes e comunidade.

A construção acontece em **5 horas efetivas** no Dia 3 (2h pela manhã + 3h pela tarde), seguida de 45 min para gravação do vídeo demo. O briefing completo acontece no fim do Dia 2 (slot 16:15–16:45), dando a você 5 dias de “marinade mental” entre Dia 2 e Dia 3 para escolher problema, corpus e arquitetura.

2. Formato em duplas

O projeto é desenvolvido em **duplas (2 alunos)**. Com 62 alunos na disciplina, isso resulta em 31 projetos. Dupla é o tamanho ótimo: cabe pair programming, divisão clara de tarefas, debug compartilhado e ainda mantém peso individual suficiente para portfólio.

2.1 Como formar a dupla

- **Antes do Dia 3:** combine sua dupla durante o intervalo do Dia 2 ou pelo chat do Teams entre Dia 2 e Dia 3. Quem chegar ao Dia 3 sem dupla é alocado pelo instrutor no kickoff (10:00).
- **Critério de afinidade:** procure alguém com **interesse em problema/corpus similar** ao seu. Heterogeneidade técnica (1 forte em backend + 1 forte em ML. é vantagem).
- **Restrições:** dupla **não pode** repetir alguém com quem você fez assignment anterior, para diversificar a experiência de pair.

2.2 Divisão de trabalho sugerida

Para terminar dentro do tempo, divida o template em duas frentes:

Aluno	Frente	Arquivos
A	Pipeline RAG + Tool-use	src/pipeline/rag.py , src/pipeline/tools.py
B	Custo + UI + Deploy	src/pipeline/cache.py , src/pipeline/routing.py , src/ui/streamlit_app.py

Ambos colaboram em: escolha do corpus, README final, gravação do vídeo, decisões de arquitetura. Pair programming nas integrações (quando RAG conversa com Tool, quando UI consome o pipeline).

2.3 Nota e responsabilidade

- A nota do projeto é **única e idêntica para os dois membros** da dupla.
- O vídeo demo deve ser **gravado em conjunto** (ambos aparecem, cada um explica sua frente de trabalho).
- Em caso de desbalanço grave de contribuição (1 aluno fez ~tudo), a dupla pode registrar isso no Forms de entrega — o instrutor avalia individualmente.

3. Critérios de aceitação

Para a rubrica avaliar, seu projeto **PRECISA** ter:

- Pelo menos 1 corpus textual com ≥ 10 páginas (livro, docs, papers, regulamentação)
- Pelo menos 3 perguntas que se beneficiam de RAG (não respondíveis sem o corpus)
- Pelo menos 1 tool customizada do domínio (function-calling de verdade, não decorativa)
- Deploy em URL pública (Streamlit Cloud / HuggingFace Spaces / Fly.io)
- README profissional (problem + arquitetura + setup + custo + decisões + limites)
- Pelo menos 1 medida de redução de custo (cache exact OU semantic OU routing)

Não conta como projeto:

- Wrapper de 1 endpoint LLM (só chama `gpt-4o-mini` em um for loop)
- Sumarizador puro (não usa retrieval)
- Chatbot genérico sem corpus específico
- Tutorial copy/paste sem personalização

4. Modalidades

Você pode escolher **uma de 2 modalidades**:

Modalidade	Quando faz sentido	Liberdade	Recomendado para trilha
A — Template + corpus próprio	A dupla quer entregar com certeza, sem retrabalho	Personaliza corpus + 1 tool específica + 6 TODOs marcados	Basic, Intermediate
B — Projeto autoral	A dupla tem um problema claro e quer construir do zero	Total — pode reusar partes do template ou criar tudo	Intermediate, Advanced

Importante: ambas usam a **mesma rubrica** (§6). Quem usa o template **não fica em desvantagem** — o que conta é a entrega final. A escolha A/B é confirmada no kickoff de 5 min na manhã do Dia 3.

4.1 Modalidade A — Template + corpus próprio (caminho seguro)

`delivery/projetos/template-portfolio/` tem scaffold completo:

- `pyproject.toml` com dependências fixadas
- `src/pipeline/{rag,tools,cache,routing}.py` com 6 TODOs marcados `# SEU CODIGO AQUI`
- `src/ui/streamlit_app.py` pronto para deploy 1-click no Streamlit Cloud
- `src/observability/trace.py` para logging estruturado
- `tests/test_smoke.py` para validar smoke test do pipeline
- `README.md` skeleton com seções esperadas + TODOs de preencher

A dupla faz:

1. Clona o template, configura `.env` com sua API key
2. Substitui corpus de `data/corpus/` pelo escolhido (livro, docs, código, etc.)
3. Preenche 6 TODOs (cada um ~20–30 min de trabalho, divididos entre A e B)
4. Customiza 1 tool específica do domínio (`tools.py` TODO 4)
5. Deploy Streamlit Cloud (5 min via GitHub)
6. Preenche README com problema, arquitetura, métricas e custo

Resultado: projeto funcional + deployado em ~3 h efetivas. Sobra ~2 h para polish e gravação.

4.2 Modalidade B — Projeto autoral

A dupla constrói tudo a partir dos notebooks 01–05. Mais liberdade arquitetural, mais risco de não terminar.

Recomendação: se for B, tenha problema e corpus alinhados antes do kickoff do Dia 3 (use os 5 dias entre Dia 2 e Dia 3 para isso, sem trabalho extra fora das horas — apenas pensar).

4.3 Não tem ideia de problema?

Veja a §5 — **5 ideias pré-validadas** com corpus indicado, scope estimado, tool sugerida e nível de dificuldade.

5. PROBLEM IDEAS — 5 ideias pré-validadas

Lista de 5 ideias **pré-validadas pelo instrutor** que cabem em ~3 h de trabalho a partir do `template-portfolio/`. A dupla pode:

- **Pegar uma direta** — instrutor já validou que scope é factível, corpus é acessível, tool faz sentido
- **Usar como inspiração** — adaptar para seu domínio
- **Ignorar** e fazer projeto autoral (Modalidade B)

Cada ideia traz: problema, corpus indicado, tool customizada sugerida, scope estimado, e nível de dificuldade.

5.1 Ideia 1 — Q&A sobre Livro Técnico

Problema: assistente que responde dúvidas sobre um livro técnico open-source, com citação de página.

Corpus (alternativas open-access):

- [“Crafting Interpreters” por Robert Nystrom](#) — HTML, conversível para PDF (~600 págs)
- [“Pro Git” por Scott Chacon](#) — CC BY-NC-SA, ~440 págs PDF
- [“The Linux Command Line” por William Shotts](#) — Creative Commons

Tool sugerida:

```
def lookup_chapter(chapter: int) -> str:
    """Retorna o sumário do capítulo N do livro."""
```

Scope:

- Ingest do livro inteiro (chunking recursive 800/100)
- 5 perguntas de teste cobrindo capítulos diferentes
- Tool `lookup_chapter` para navegação dirigida

Dificuldade: ☆ Básico — corpus claro, tool simples, cabe em 3 h confortável.

Recomendado para trilha: Basic, Intermediate.

5.2 Ideia 2 — Q&A sobre Changelog de uma Library

Problema: “Quando essa feature foi adicionada?” / “Esse comportamento mudou na v2.x?” sobre uma lib popular.

Corpus: changelog + release notes de uma lib que você usa:

- `fastapi` [CHANGELOG.md](#) — markdown longo, fácil parse
- `pydantic` [CHANGELOG](#) — releases granulares com diff
- `django` [release notes](#) — múltiplos `.txt/.md` por versão

Para baixar: `git clone --depth 1 <repo> && cd <repo> && cat CHANGELOG.md > changelog.txt`

Tool sugerida:

```
def check_compat(lib: str, from_version: str, to_version: str) -> dict:
    """Retorna lista de breaking changes entre 2 versões."""
```

Scope:

- Ingest do changelog (1 doc grande)

- Chunking por versão (separadores: `## v` ou `### Version`)
- Tool que filtra chunks por range de versão

Dificuldade: ☆☆ Intermediário — corpus estruturado mas chunking não-trivial; tool útil de verdade.

Recomendado para trilha: Intermediate.

5.3 Ideia 3 — Resumo Buscável de Podcast/Vídeo

Problema: você ouviu 5 h de podcast técnico por semana; quer fazer Q&A em transcripts e localizar o trecho exato.

Corpus: transcripts de podcasts/vídeos que você já consome:

- [Lex Fridman Podcast transcripts](#) — disponíveis no site (ou via YouTube `youtube-transcript-api`)
- [The Changelog](#) — transcripts oficiais por episódio
- Transcripts gerados via [Whisper](#) de qualquer áudio próprio

Para baixar transcripts do YouTube:

```
pip install youtube-transcript-api
python -c "from youtube_transcript_api import YouTubeTranscriptApi;
print(YouTubeTranscriptApi.get_transcript('VIDEO_ID', languages=['pt','en']))"
```

Tool sugerida:

```
def get_timestamp(quote: str) -> dict:
    """Retorna timestamp (mm:ss) e URL do trecho onde a frase aparece."""
```

Scope:

- 3-5 transcripts de episódios (~30 k tokens cada)
- Chunks com metadata de timestamp (preservar `start_time` no metadata do Chroma)
- Tool retorna URL com `?t=Xs` (YouTube) ou referência ao timestamp

Dificuldade: ☆☆☆ Avançado — metadata enriquecido + tool com side-effect (linkar áudio/vídeo).

Recomendado para trilha: Intermediate, Advanced.

5.4 Ideia 4 — Code Reviewer Especializado

Problema: “Esse código está seguindo as convenções do projeto X?”

Corpus: style guides + lint rules + decisões arquiteturais de um projeto open-source:

- [Google Python Style Guide](#)
- [Django coding style](#)
- [ADRs de algum projeto](#) — decisões arquiteturais como corpus

OU corpus próprio:

- ADRs do seu time no trabalho (com permissão)
- CONTRIBUTING.md + style guide do seu repo pessoal

Tool sugerida:

```
def run_linter(code: str) -> str:
    """Roda `ruff check` no snippet e retorna lista de erros."""
```

⚠ **Sandboxing:** `subprocess.run(['ruff', 'check', '--stdin-filename=tmp.py', '-'], input=code, ...)`. **Não use `eval` direto no código do usuário.**

Scope:

- Ingest dos style guides

- Pergunta tipo “este pedaço de código viola alguma regra?”
- Tool `run_linter` para validação determinística

Dificuldade: ★★★ Avançado — tool com `subprocess` precisa de cuidado de segurança; ainda assim factível em 3 h.

Recomendado para trilha: Advanced.

5.5 Ideia 5 — Assistente de Compliance LGPD

Problema: dev pergunta “Posso armazenar X dado dessa forma?” e o assistente responde citando artigo da LGPD.

Corpus: texto integral da LGPD + ANPD orientações:

- [LGPD — Lei 13.709/2018 \(texto integral\)](#) — copiar para PDF/MD
- [ANPD — Guias e Orientações](#)
- [Lei 13.709/2018 PDF oficial](#)

Tool sugerida:

```
def cite_article(article_number: int) -> str:
    """Retorna texto integral do Art. N da LGPD."""
```

Scope:

- Ingest da LGPD + 1–2 guias da ANPD (~100 págs total)
- 5 cenários típicos: armazenamento de CPF, retenção de dados, base legal
- Tool `cite_article` para evitar alucinação de artigo (LLM tende a inventar números)

Dificuldade: ★★ Intermediário — corpus jurídico denso, mas curto; tool simples; aplicação real para o Brasil.

Recomendado para trilha: Intermediate.

5.6 Comparativo rápido

#	Ideia	Corpus	Tool	Dificuldade	Trilha
1	Q&A livro técnico	open-source book (Pro Git, Crafting Interpreters)	<code>lookup_chapter</code>	★	Basic
2	Changelog QA	CHANGELOG.md de lib popular	<code>check_compat</code>	★★	Intermediate
3	Podcast searchable	YouTube transcripts	<code>get_timestamp</code>	★★★	Intermediate/Advanced
4	Code reviewer	Style guides + ADRs	<code>run_linter</code> (sandboxed)	★★★	Advanced
5	Compliance LGPD	LGPD + ANPD guias	<code>cite_article</code>	★★	Intermediate

5.7 Como escolher

1. **Tem corpus que a dupla já conhece?** → Ideia 1 (livro) ou 5 (LGPD)
2. **Quer algo que vai usar depois do curso?** → Ideia 2 (changelog do que usam) ou 3 (seus podcasts)
3. **Quer mostrar capacidade técnica avançada?** → Ideia 3 ou 4
4. **Quer entregar com certeza no tempo?** → Ideia 1 ou 5
5. **Nenhuma combina?** → Use como inspiração, faça projeto autoral

6. Estrutura do `template-portfolio`

```

template-portfolio/
├── pyproject.toml           # dependências fixadas (uv)
├── .env.example            # template de variáveis (API keys)
├── .gitignore
├── README.md              # skeleton (preencher problema, arq., métricas)
├── data/
│   └── corpus/
│       └── README.md      # substituir pelo seu corpus
├── src/
│   ├── pipeline/
│   │   ├── rag.py         # TODO 1: chunking + embedding + retrieval
│   │   ├── tools.py       # TODO 2/4: tool genérica + tool do domínio
│   │   ├── cache.py       # TODO 3: cache semântico
│   │   └── routing.py     # TODO 5: model routing cheap-first
│   ├── ui/
│   │   ├── streamlit_app.py # TODO 6: chat UI + streaming
│   │   └── observability/
│   │       └── trace.py    # logging estruturado (já pronto)
└── tests/
    └── test_smoke.py      # smoke test do pipeline

```

Os 6 TODOs estão marcados `# SEU CODIGO AQUI` no código. Cada um leva ~20–30 min para um desenvolvedor focado.

7. Rubrica (3 bandas, 100 pts ponderados)

Critério	Peso	Básico (60–74)	Sólido (75–89)	Excelente (90–100)
Técnica	40%	LLM + RAG + tool-use funcionam isoladamente	+ integração ponta-a-ponta com erros tratados	+ arquitetura justificada + código testado + eval automatizada com RAGAS
README	30%	problem statement + setup	+ diagrama de arquitetura + métricas observadas	+ decisões de design explicadas + GIF de demo + custo por requisição reportado
Custo/Latência	20%	mede custo por chamada	+ cache implementado com hit-rate medido	+ routing cheap-first com redução ≥50% + P95 de latência reportado
Demo	10%	URL acessível	+ 1 fluxo demonstrável sem crash	+ UX cuidada (streaming, loading states) + TTL de cache declarado

Diferenciação por trilha adaptativa (não muda rubrica, muda target esperado):

- Trilha **Basic** — target “básico” em todos os 4 critérios
- Trilha **Intermediate** — target “sólido” em ≥3 critérios
- Trilha **Advanced** — target “excelente” em ≥2 critérios

7.1 Faixas de nota

Score final	Conceito	Significado
85–100	A	Excelente — domínio completo, projeto apto a portfólio público
70–84	B	Bom — projeto sólido com 1 critério em “básico”
55–69	C	Básico — projeto funcional mas com gaps notáveis
0–54	D	Insuficiente — revisar notebooks 01–06 e refazer

8. Entrega — 3 partes, todas via Forms

A entrega acontece no slot **16:00–16:45 do Dia 3**, durante o horário de aula. Não há janela de entrega fora desse horário.

Partes da entrega:

1. **URL pública da demo** — Streamlit Cloud / HuggingFace Spaces / Fly.io
2. **URL do repositório** — GitHub público com README profissional + GIF de demo
3. **URL do vídeo demo (≤ 3 min)** — gravado em Loom / MS Stream / OBS / Teams; conteúdo:
 - Aluno A apresenta o problema e o corpus escolhido (30 s)
 - Aluno A demonstra o fluxo RAG + tool-use funcionando (60 s)
 - Aluno B mostra a redução de custo (cache hit-rate / model routing) (45 s)
 - Aluno B comenta uma decisão de design e os limites do projeto (45 s)

O vídeo substitui a apresentação ao vivo individual (inviável com 31 duplas em 1 h). O instrutor projeta uma amostra de 3–4 demos ao vivo no encerramento (16:45–17:00) como fechamento da disciplina.

8.1 Como enviar pelo Forms

1. **Suba o vídeo** em Loom, MS Stream ou YouTube *unlisted* — pegue o link compartilhável
2. **Suba o código** no GitHub (público) — pegue o URL do repo
3. **Confirme o deploy** abrindo a URL pública numa janela anônima
4. Abra o **Forms de Entrega do Projeto** (link no chat às 13:00 do Dia 3)
5. Cole as 3 URLs nos campos correspondentes
6. **Identifique a dupla:** nome dos 2 alunos + email institucional de ambos
7. Envie — pode reenviar dentro da janela; vale o último envio

⚠ Antes de submeter, **confirme** que todos os 3 links abrem numa janela anônima.

9. Plantão durante o Dia 3

Situação	O que fazer
Dúvida rápida (import, path, comando)	Chat público do Teams; preceptor responde
Travou >10 min no mesmo TODO	Pulem para o próximo, voltem depois (especialmente TODO 5/6)
Erro de deploy no Streamlit	Breakout “Plantão de Deploy” — instrutor te ajuda 1:1
Rate limit no Gemini free tier	Distribuem chamadas (15 RPM = 1 a cada 4 s) ou troquem para OpenAI \$5 free credit
API key vazou em commit	Avise o instrutor imediatamente — revoguem a key e gerem nova

10. Política de plágio e uso de IA

- **Plágio entre duplas:** projetos idênticos (sem variação relevante) recebem **zero para ambos**. Duplas podem discutir ideias entre si, mas o código é da dupla.
- **Uso de IA na entrega: permitido e esperado** (é uma disciplina sobre LLMs). Mas: a dupla deve **entender** o código entregue — pode haver Q&A oral no encerramento.
- **Revisão de nota:** até 3 dias após a divulgação, com justificativa técnica.

11. Checklist final antes de enviar

Repo no GitHub é **público** (abre numa janela anônima)

URL da demo abre sem login e o fluxo principal funciona

README contém: problema, setup, arquitetura, métricas, custo, decisões, limites

README tem GIF (ou screenshot) da demo funcionando

Vídeo demo tem ≤ 3 min e ambos os alunos aparecem/falam

Pelo menos 1 medida de redução de custo está reportada (hit-rate de cache OU routing cheap-first)

Forms enviado com 3 URLs + nomes da dupla + emails institucionais

Bom projeto. Em caso de bloqueio crítico, **avise cedo no chat** — o preceptor e o instrutor estão de plantão durante todo o Dia 3.